

MAREVLO · PRODUCTION RAG

Observability

Seeing inside the black box — so you can finally tell why an answer was wrong, and prove when you have made it better.

PART II — MAKING IT WORK

Module 8

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

We ended the last module with an uncomfortable admission: our RAG system works, but it is a black box. When it gives a wrong answer, we cannot see why. When it is slow, we cannot see where the time went. When we change something, we cannot tell whether we made it better or quietly made it worse. This module is the cure. It is about **observability** — the practice of making a system show you what it is doing, so that you can understand it, fix it, and improve it on purpose rather than by luck.

This is one of the most important modules in the course, even though it adds no new "feature" a user would ever see. The reason is simple and worth sitting with: *you cannot improve what you cannot see, and you cannot prove an improvement you cannot measure*. Almost every other problem in RAG becomes far harder to solve, or impossible, when you are flying blind. So before we try to optimise anything in the next module, we first give ourselves eyes.

THE WHOLE MODULE IN ONE SENTENCE

Observability has three pillars — *tracing* (seeing one request in full detail), *metrics* (tracking the system's health over time), and *feedback* (capturing what real users got and whether it helped) — and together they let you build an *evaluation set*, the fixed test that turns guessing into measuring.

Where we are

We are not changing how the system answers questions in this module. We are adding the ability to *watch* it answer them. Everything we build here — traces, metrics, feedback capture, an evaluation set — exists to support the next module, where we finally start improving the system and need a reliable way to tell whether each change actually helped.

SECTION 1

Why a RAG system is so hard to fix blind

A single request passes through several stages, and any one of them can be the culprit. Without visibility, you are guessing which one.

Recall the journey of a single question through our system. First the question is turned into an embedding. Then the vector database is searched for the nearest chunks. Then those chunks are assembled into a prompt. Then the model reads the prompt and generates an answer. That is at least four distinct stages, and a wrong answer could come from any of them — or from the documents themselves being wrong.

Here is the trap that wastes so much time. A wrong answer usually *looks* the same no matter which stage caused it: the user just sees a bad answer. But the fix is completely different depending on the cause. If the right chunk was never retrieved, no amount of prompt tweaking will help — the model never had the information. If the right chunk *was* retrieved but the prompt was poorly written, then the prompt is exactly what to fix. Without a way to see what actually happened inside that one request, you cannot tell these apart, so you end up changing things at random and hoping. That random guessing is the single most expensive habit in RAG work, and observability is what replaces it with knowing.

THINK ABOUT IT

A user reports: "I asked about the warranty and got a wrong answer." What is the very first thing you would want to know? Not "what should I change in the prompt?" — that is jumping to a fix before you have a diagnosis. The first thing you want is to *see what the system actually retrieved for that question*. If the warranty chunk is not in there, it is a retrieval problem. If it is in there, it is a prompt or model problem. Observability is what makes that first, essential look possible.

PILLAR 1 OF 3

Tracing — seeing one request in full

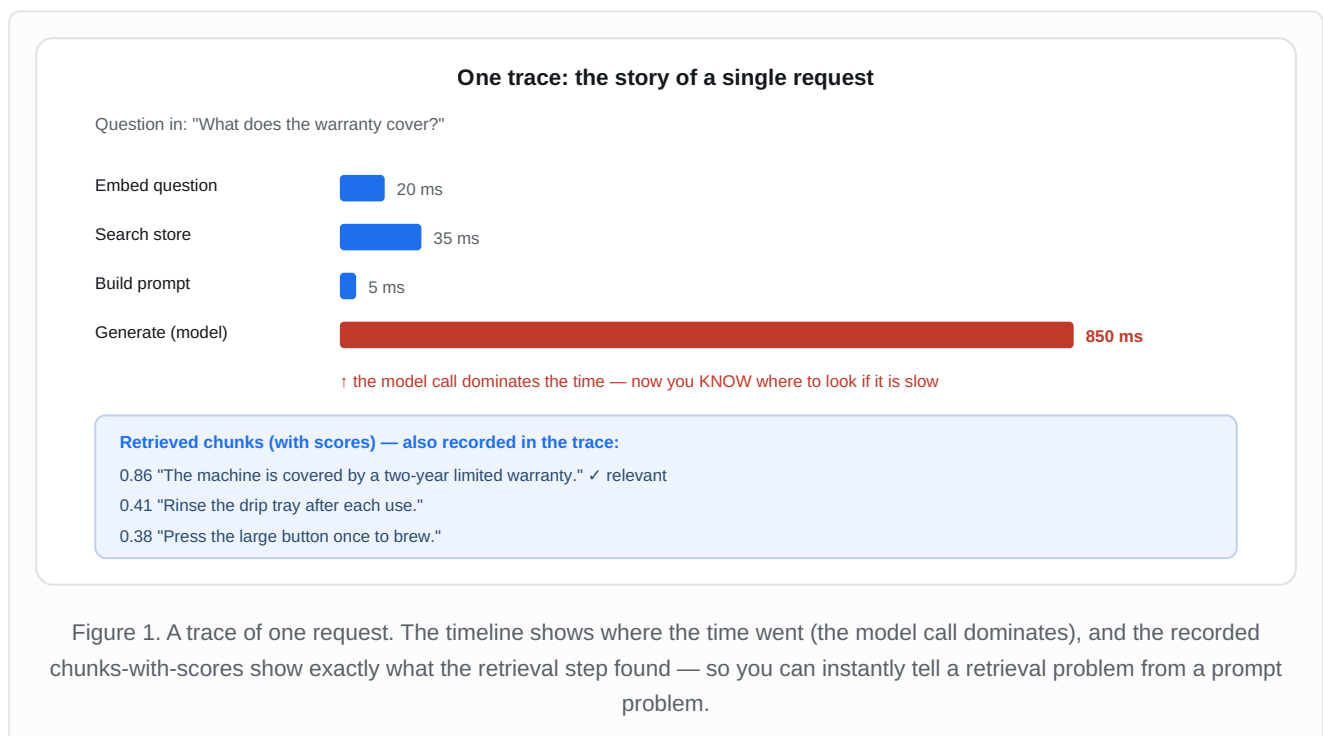
A trace is the complete, replayable story of a single request, stage by stage.

A **trace** records everything that happened during one request: the question that came in, the chunks that were retrieved and their scores, the exact prompt that was assembled, the answer the model produced, and how long each step took. With a trace in hand, the question "which stage failed?" goes from an afternoon of guessing to a ten-second look.

ANALOGY — THE FLIGHT RECORDER

An aircraft carries a flight recorder — a "black box" that captures everything happening moment to moment. When something goes wrong, investigators do not guess; they replay the recording and see exactly what happened and when. A trace is the flight recorder for one request through your RAG system. When an answer is bad, you do not theorise — you open the trace and watch the request unfold.

Let us make this concrete by looking at what a single trace actually contains, drawn as a timeline so you can see both *what* happened and *how long* each part took.



Notice the two kinds of information a good trace holds. The *timing* tells you where the time goes — here the model call dwarfs everything else, so if this request were too slow, you would know immediately not to waste effort optimising the search. The *content* — especially the retrieved chunks and their scores — tells you what the system actually had to work with. If the warranty answer were wrong, one glance at those chunks would tell you whether the warranty information was even retrieved. That single capability resolves the majority of RAG bugs.

LAB 8.1 — A mini-tracer, written by hand (so you understand what a trace IS)

```
import time

def answer_with_trace(question):
    """Answer a question AND record a trace of every stage.

    Real tools (next lab) capture this automatically and add a nice UI.
    We build a tiny version by hand first, so a 'trace' is never mysterious:
    it is simply a record of each step's inputs, outputs, and timing.
    """
    trace = {"question": question, "steps": []}

    def step(name, func):
        """Run one stage, timing it and recording what it produced."""
        start = time.time()
        result = func()
        trace["steps"].append({
            "stage": name,
            "ms": round((time.time() - start) * 1000),
            "output_preview": str(result)[:120], # a peek, for the record
        })
        return result

    # Each stage runs through step(), so each is timed and recorded.
    chunks = step("retrieve", lambda: get_relevant_chunks(question))
    trace["retrieved"] = [(round(c.metadata.get("score", 0), 2),
                           c.page_content) for c in chunks] # keep chunks+scores

    if not chunks:
        trace["answer"] = "I don't have that information."
        return trace

    prompt = step("build_prompt", lambda: build_prompt(question, chunks))
    answer = step("generate", lambda: generate_answer(prompt))
    trace["answer"] = answer
    return trace

# Now every answer comes with a full, inspectable record.
trace = answer_with_trace("What does the warranty cover?")
print(trace["retrieved"]) # did we even retrieve the warranty chunk?
print(trace["steps"])     # where did the time go?
```

That hand-built tracer is deliberately simple, but it already gives us the one thing we most needed: for any request, we can see what was retrieved and where the time went. In real systems you would not build this yourself —

dedicated tools do it automatically and add a searchable interface — but building it once removes all the mystery from the word "trace."

Metrics — the system's health over time

A trace shows one request. Metrics show the pattern across thousands — and reveal problems no single trace would.

Tracing zooms all the way in on a single request. **Metrics** do the opposite: they zoom out to show the health of the whole system over time. How fast are answers, in general? How much is each query costing? How often does retrieval find a good match? How often do errors occur? These aggregate numbers reveal trends and problems that no single trace could — a slow creep in latency, a rising cost, a drop in retrieval quality after a change.

There is one idea about metrics that is so commonly misunderstood, and so important, that we are going to spend real time on it: the difference between an *average* and a *percentile*. Getting this right is the difference between a metric that reassures you falsely and one that tells you the truth.

Why the average lies, and percentiles tell the truth

Suppose you measure how long your system takes to answer, over a thousand requests. The obvious summary is the average. But the average has a dangerous habit: it can look perfectly healthy while a meaningful slice of your users are having a terrible experience. Imagine that 950 requests were fast and 50 were extremely slow. The average might still look acceptable, because the many fast requests pull it down — and so you never notice the 50 users who waited far too long and left angry.

Percentiles fix this. To understand them, imagine lining up all thousand requests from fastest to slowest. The 50th percentile, written p50, is the one in the middle — half were faster, half slower. The 95th percentile, p95, is the point below which 95% of requests fall; only the slowest 5% are above it. The 99th percentile, p99, captures all but the slowest 1%. The slow tail that the average hid is exactly what p95 and p99 reveal.

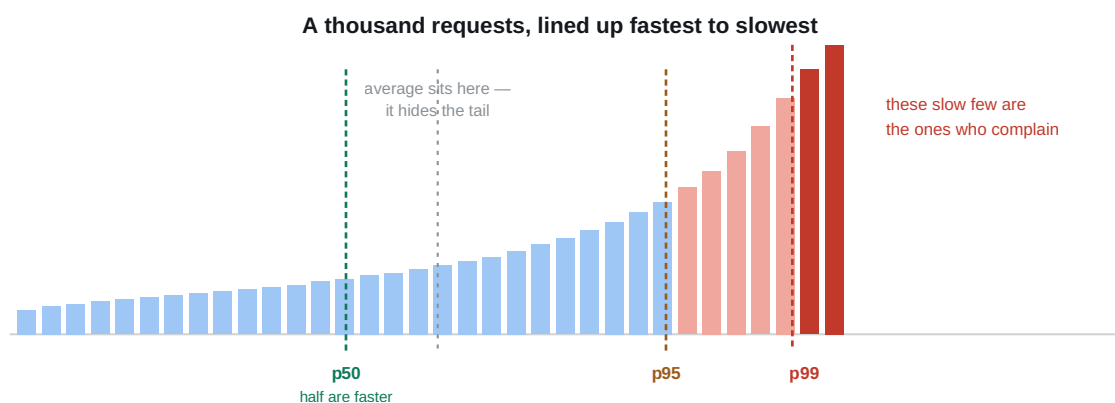


Figure 2. Most requests are fast (blue), but a slow tail (red) is where unhappy users live. The average sits comfortably low and hides that tail; p95 and p99 expose it. Watch the percentiles, not just the average.

ANALOGY — THE RESTAURANT KITCHEN

A kitchen serves most meals in ten minutes, but a few unlucky tables wait an hour. The *average* wait might be twelve minutes, which sounds wonderful on a report. But the customers who waited an hour are furious, and they are real. The p99 wait time is the number that tells you about *them*. Manage your kitchen by the average and you will keep being surprised by angry reviews; manage it by p99 and you see the problem the average was hiding.

So the metrics worth tracking for a RAG system include latency as percentiles (p50, p95, and especially p99, because that is what your unhappiest users feel), cost per query (which compounds quickly at scale), the error rate, and a measure of retrieval quality such as how often the top chunk clears your relevance threshold. Below is our own small helper to compute latency percentiles from a list of recorded times.

LAB 8.2 — Compute latency percentiles (our own helper)

```
def percentile(values, p):
    """Return the value below which p percent of the sorted values fall.

    Example: percentile(times, 99) is the p99 latency -- only the slowest
    1 percent of requests were worse than this.
    """
    if not values:
        return 0
    ordered = sorted(values)
    # Find the position p% of the way through the sorted list.
    index = min(int(len(ordered) * p / 100), len(ordered) - 1)
    return ordered[index]

# Suppose we collected the response time of every request in a list.
latencies_ms = [collect_latency() for _ in range(1000)]

print("p50:", percentile(latencies_ms, 50), "ms") # the typical request
print("p95:", percentile(latencies_ms, 95), "ms") # a slow-ish request
print("p99:", percentile(latencies_ms, 99), "ms") # the unhappy tail
```

Feedback — learning from real users

Every real question your system answers is free data about what works and what fails — if you capture it.

The third pillar is the simplest to describe and the most often neglected. Every time a real user asks your system something, you have a chance to learn: what did they ask, what answer did they get, and was it good? If you capture this — even just a thumbs-up or thumbs-down on each answer — you build, for free, the single most valuable asset in the whole improvement loop: a growing record of real questions and which ones your system handled well or badly.

This matters because the questions you imagine while building are never the questions real users actually ask. Real users phrase things strangely, ask about edge cases you forgot, and probe the gaps in your documents. Their failures are gold: each thumbs-down is a real example of where your system falls short, and a real example is worth more than any number of hypothetical ones. We will use exactly this record in the next section to build a proper test.

A brief but important word of care here. When you log real user questions and answers, you may be capturing personal or sensitive information. Be deliberate about what you store, who can see it, and how long you keep it — logging is not an excuse to hoard private data, and a careless log can become a privacy problem of its own. Capture what you need to improve the system, and no more.

LAB 8.3 — Capture feedback on each answer (our own, minimal)

```
import json, time

def log_interaction(question, answer, retrieved, helpful=None):
    """Record one real interaction so we can learn from it later.

    `helpful` is the user's thumbs up/down (True/False) when they give it.
    Note: be mindful of privacy -- store only what you need to improve.
    """
    record = {
        "time": time.time(),
        "question": question,
        "answer": answer,
        "retrieved": [c.page_content for c in retrieved], # what it had to work with
        "helpful": helpful,                             # the feedback signal
    }
    with open("interactions.log", "a") as f:
        f.write(json.dumps(record) + "\n") # one record per line

# Later, the thumbs-down records are your richest source of real failures.
```

THE PAYOFF

From feedback to an evaluation set: turning guessing into measuring

The three pillars let you see and capture. The evaluation set lets you prove whether a change actually helped.

Here is where everything comes together. Once you can capture real questions and judge whether answers were good, you can build an **evaluation set**: a fixed list of questions, each paired with the answer you know to be correct. You run your system against this fixed list and score how many it gets right. That score is your baseline. Now, whenever you change anything — the chunk size, the prompt, the search method — you run the evaluation set again and compare. If the score went up, the change helped. If it went down, you just learned that before your users did.

ANALOGY — THE FIXED EXAM

Imagine trying to tell whether students are improving while changing the exam every week. You could not — a higher score might just mean an easier exam. The only way to measure real progress is a *fixed* exam: same questions, scored the same way, before and after teaching. An evaluation set is that fixed exam for your RAG system. Without it, every "improvement" is judged on a different question on a different day, which is to say, judged on nothing.

THE HARD TRUTH THIS SECTION EXISTS TO DELIVER

Without a fixed evaluation set, optimisation is superstition. You will change something, try a question or two that happen to work, declare victory, and have no idea that you quietly broke ten other questions. Teams do this constantly, and it is why their systems wander sideways for months. The evaluation set is what makes the next module — optimisation — real work instead of guesswork. **HIGH CONFIDENCE**

LAB 8.4 — A simple evaluation runner (our own)

```
# A fixed set of questions with known-good answers. Seed it from real
# user failures (Pillar 3), not just easy questions you made up.
eval_set = [
    {"question": "How often should I descale?", "expect": "two months"},
    {"question": "What does the warranty cover?", "expect": "two-year"},
    {"question": "How do I clean the drip tray?", "expect": "rinse"},
    # ... grow this to 30+ cases, including ones the system has failed.
]

def run_eval(eval_set):
    """Score the system against the fixed set. Returns the fraction passed.

    Here we use a simple check: does the expected phrase appear in the
    answer? This is crude but repeatable -- and repeatable is the point.
    (More careful scoring can use a model as a judge; see Module 9.)
    """
    passed = 0
    for case in eval_set:
        answer = answer_question(case["question"])
        if case["expect"].lower() in answer.lower():
            passed += 1
        else:
            print(f"FAILED: {case['question']!r} -> {answer!r}")
    score = passed / len(eval_set)
    print(f"Score: {passed}/{len(eval_set)} = {score:.0%}")
    return score

baseline = run_eval(eval_set) # run BEFORE any change, to compare against
```

IN PRACTICE

Using a real observability tool

You will not hand-build all of this in production. Dedicated tools capture traces automatically and give you a searchable interface.

We built small versions of these tools by hand so that none of the ideas would feel like magic — you now know exactly what a trace, a percentile, and an evaluation run really are. In a real system, you would lean on a dedicated observability tool that captures traces automatically and presents them in a searchable interface, so that finding the trace for a specific bad answer takes seconds. LangSmith is the tool built specifically for systems made with the libraries we have been using; once enabled, it records a full trace of every request without you changing your answering code at all. The principle is the important part, and it is the same whichever tool you choose: capture everything about each request, so that when something goes wrong you can look instead of guess.

LAB 8.5 — Turning on automatic tracing (conceptual)

```
import os

# With these settings in place, a tracing tool records every request
# through the libraries automatically -- no change to your answer code.
# Keep secrets in a secret manager, not written directly in your code.
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = get_secret("LANGSMITH_API_KEY")

# From here on, calls through the framework appear as full traces in the
# tool's interface: question, retrieved chunks, prompt, answer, timing.
```

FAULTS AND CORRECTIONS

The observability mistakes that keep systems broken

For each: the broken habit, why it leaves you blind, and the fix.

FAULT 1 — No tracing, so bugs cannot be reproduced

THE BROKEN HABIT

```
# The system answers and forgets. A user reports a bad answer and there  
# is no record of what was retrieved or what prompt was sent.  
answer = answer_question(question) # nothing recorded
```

WHY IT LEAVES YOU BLIND

When a bad answer is reported, you have nothing to look at. You cannot see what was retrieved, so you cannot tell a retrieval failure from a prompt failure, and you cannot even reproduce the problem reliably. Every debugging session starts from zero, and the same bugs recur because you never saw their cause.

THE CORRECTION

```
# Trace every request -- by hand (Lab 8.1) or with a tool (Lab 8.5).  
trace = answer_with_trace(question) # now every answer is inspectable
```

FAULT 2 — Watching only the average, missing the slow tail

THE BROKEN HABIT

```
# Reporting only the average response time. It looks healthy while a  
# slice of users suffer long waits the average quietly absorbs.  
print("avg latency:", sum(latencies) / len(latencies))
```

WHY IT LEAVES YOU BLIND

The average is pulled down by the many fast requests, so it can look fine while your slowest 1–5% of users are having an awful experience. Those are precisely the users who complain and leave, and the average is structurally incapable of showing them to you. You end up surprised by anger your dashboard said could not exist.

THE CORRECTION

```
# Track percentiles, especially p95 and p99 -- the tail users feel.  
print("p50:", percentile(latencies, 50))  
print("p95:", percentile(latencies, 95))  
print("p99:", percentile(latencies, 99)) # the truth the average hid
```

FAULT 3 — Logging the answer but not the retrieved chunks

THE BROKEN HABIT

```
# We save the question and the answer, but not what was retrieved.
log = {"question": question, "answer": answer} # the crucial middle is missing
```

WHY IT LEAVES YOU BLIND

The retrieved chunks are the single most useful thing to record, because they are what separate a retrieval failure from a generation failure — the most common fork in all of RAG debugging. Without them, a wrong answer is unsolvable: you cannot tell whether the right information was never found, or was found and then mishandled. You logged the symptom and threw away the evidence.

THE CORRECTION

```
# Always record the retrieved chunks (and their scores) too.
log = {"question": question, "answer": answer,
      "retrieved": [c.page_content for c in chunks]}
```

FAULT 4 — No evaluation set, so improvement is judged by feel

THE BROKEN HABIT

```
# Change something, try one question that happens to work, ship it.
# No fixed test, so 'better' is just a feeling.
new_prompt = tweak(prompt)
print(answer_question("the one question I keep trying")) # looks good?
```

WHY IT LEAVES YOU BLIND

Judging a change on one or two questions you happen to try is not measurement; it is anecdote. The change may fix that question and silently break ten others you did not check. Without a fixed set scored the same way every time, you have no honest way to know whether you are moving forward or sideways — and most teams without an eval set spend months going sideways.

THE CORRECTION

```
# Score against a fixed eval set before and after every change.
before = run_eval(eval_set)
new_prompt = tweak(prompt)
after = run_eval(eval_set)
print("change helped" if after > before else "change hurt -- revert")
```

FAULT 5 — Changing several things at once

THE BROKEN HABIT

```
# Change the chunk size, the prompt, AND the search method together,  
# then run the eval. The score moved -- but which change caused it?  
adjust_chunking(); rewrite_prompt(); switch_to_hybrid()  
run_eval(eval_set) # result is uninterpretable
```

WHY IT LEAVES YOU BLIND

If you change three things and the score moves, you cannot tell which change helped, which hurt, or whether two of them cancelled out. You learn nothing you can build on, and you may keep a harmful change because a helpful one masked it. An evaluation set only gives clear answers when you ask it one clear question at a time.

THE CORRECTION

```
# Change ONE thing, measure, keep or revert, then change the next.  
before = run_eval(eval_set)  
adjust_chunking()  
after = run_eval(eval_set) # now the score change is attributable
```

FAULT 6 — An evaluation set of only easy, made-up questions

THE BROKEN HABIT

```
# An eval set of gentle questions you invented, all of which already pass.  
eval_set = [{"question": "How do I clean it?", "expect": "wipe"}] # too easy
```

WHY IT LEAVES YOU BLIND

An evaluation set made only of easy questions gives you a comfortable, meaningless high score. It tells you nothing about the hard, strange, real questions where your system actually struggles, so a green dashboard sits happily on top of a system that fails real users. The whole value of the set is in the questions it can *fail*.

THE CORRECTION

```
# Seed the eval set from REAL failures captured by feedback (Pillar 3).  
# Every thumbs-down becomes a test case, so the set tests what actually  
# breaks -- not what you hoped would pass.  
hard_cases = load_thumbs_down("interactions.log")  
eval_set.extend(hard_cases)
```

Checkpoint: try it yourself

This exercise gives you the core habit of the whole module — using a trace to diagnose, and an eval set to measure — in about thirty minutes.

1. Wrap your system with the mini-tracer from Lab 8.1. Ask it three questions and, for each, look at the recorded chunks and timings. Notice how the trace immediately tells you what was retrieved and where the time went.
2. Deliberately cause a retrieval failure: ask a question whose answer is not in your documents. Open the trace and confirm you can see — directly, not by guessing — that the right chunk simply was not retrieved. That is the retrieval-versus-generation diagnosis made visible.
3. Build a small evaluation set of five questions with known answers, including at least one your system currently gets wrong. Run `run_eval` and record the baseline score.
4. Now change one thing — for example, raise the number of retrieved chunks — and run the eval again. Compare the scores. Whether it went up or down, notice that you now *know*, rather than guess, what the change did.

WHAT YOU SHOULD NOTICE

In step 2, you did not theorise about why an answer was wrong — you opened the trace and saw it. In step 4, you did not feel that a change was better — you measured it. That shift, from theorising and feeling to seeing and measuring, is the entire point of observability, and it is what makes the next module possible.

What to carry into Module 9

THE SIX THINGS TO REMEMBER

One, you cannot improve what you cannot see, or prove an improvement you cannot measure — observability is the foundation of all real progress. **Two**, a trace is the full, replayable story of one request; recording the retrieved chunks is the most valuable part, because it separates retrieval failures from generation failures. **Three**, metrics track health over time — and you must watch percentiles, especially p99, not just the average, which hides the slow tail your unhappiest users feel. **Four**, capture feedback on real answers; every thumbs-down is a real failure worth more than any imagined one. **Five**, turn that feedback into a fixed evaluation set — the exam that lets you score before and after a change. **Six**, without an eval set, optimisation is superstition; with one, change exactly one thing at a time and measure.

Now that we can see inside the system and measure it honestly, we are finally ready to make it better — on purpose. Module 9 is about optimisation: the specific techniques that improve retrieval quality, applied in the right order, each one kept or discarded based on whether it raises the score on the evaluation set we just built. Everything we made here was so that the next module can be real engineering instead of guesswork.